Brute Force and Backtracking Algorithms

Brute Force Algorithms

- Solves a problem by traversing the entire search space in the worst case and enumerates all possible solutions: feasible and unfeasible .
- It traverses a large number of solutions➔ exponential runtime ➔ not a good option for a large **N**
- This method should be the last option when no other method is known to solve the problem
- Can be used to
  - Search for a valid solution or enumerate solutions
  - Solve an optimization problem by finding the optimal solution

Brute Force : Ex 1 Enumerate Solutions

```
Global
int N
Define integer array solution[N]
```

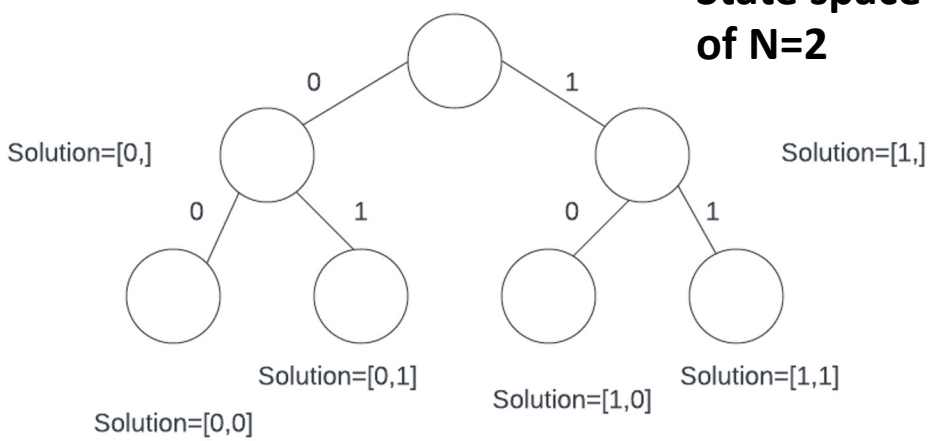
```
print()
   for i = 0 to N-1
         print solution[i]
```

```
void solve(int i)
   if (i == N)
      print()
      return

   solution[i] = 0
   solve(i + 1)

   solution[i] = 1
   solve(i + 1)
```

N=2
Solve(0)

Printed Solution
00
01
10
11

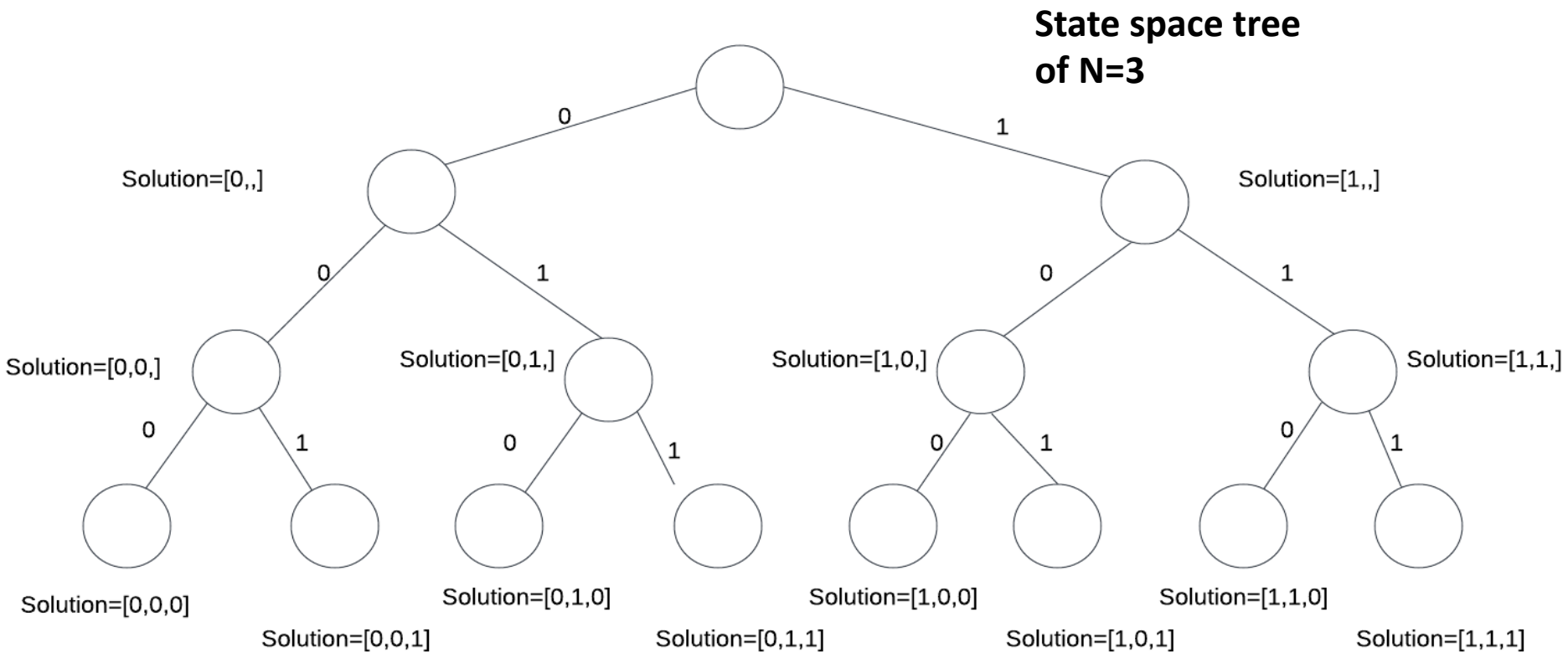
State space tree of N=2



Brute Force Algorithms: Enumerate Solutions

N=3
Solve(0)

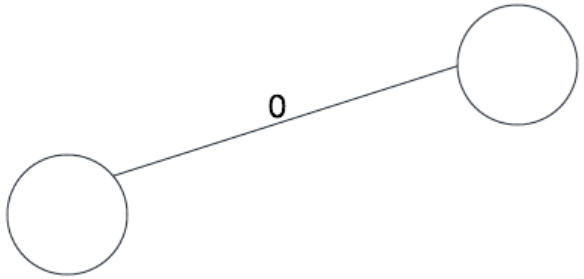
Printed Solution
000
001
010
011
100
101
110
111

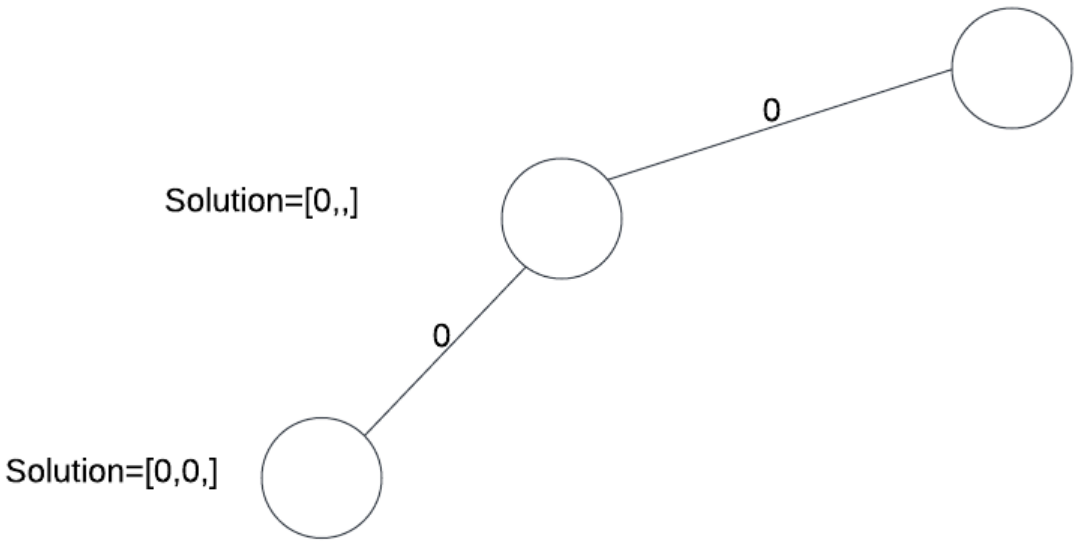


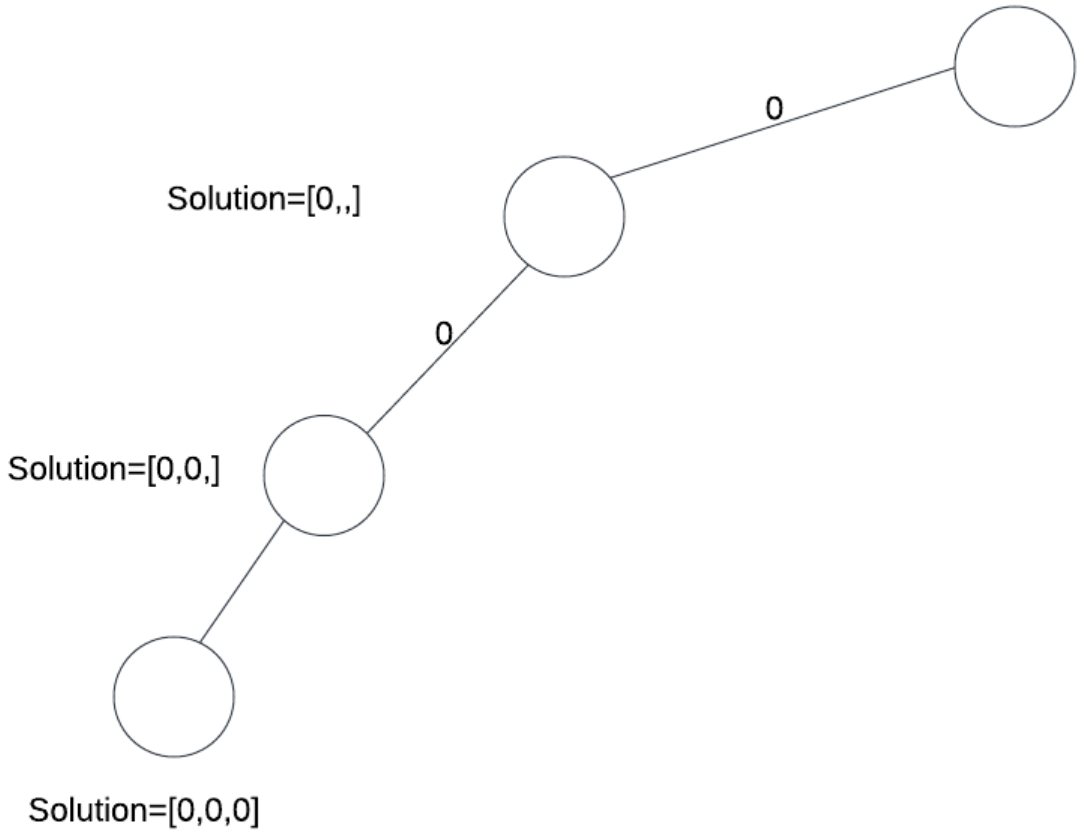
Number solutions to explore = 2^N

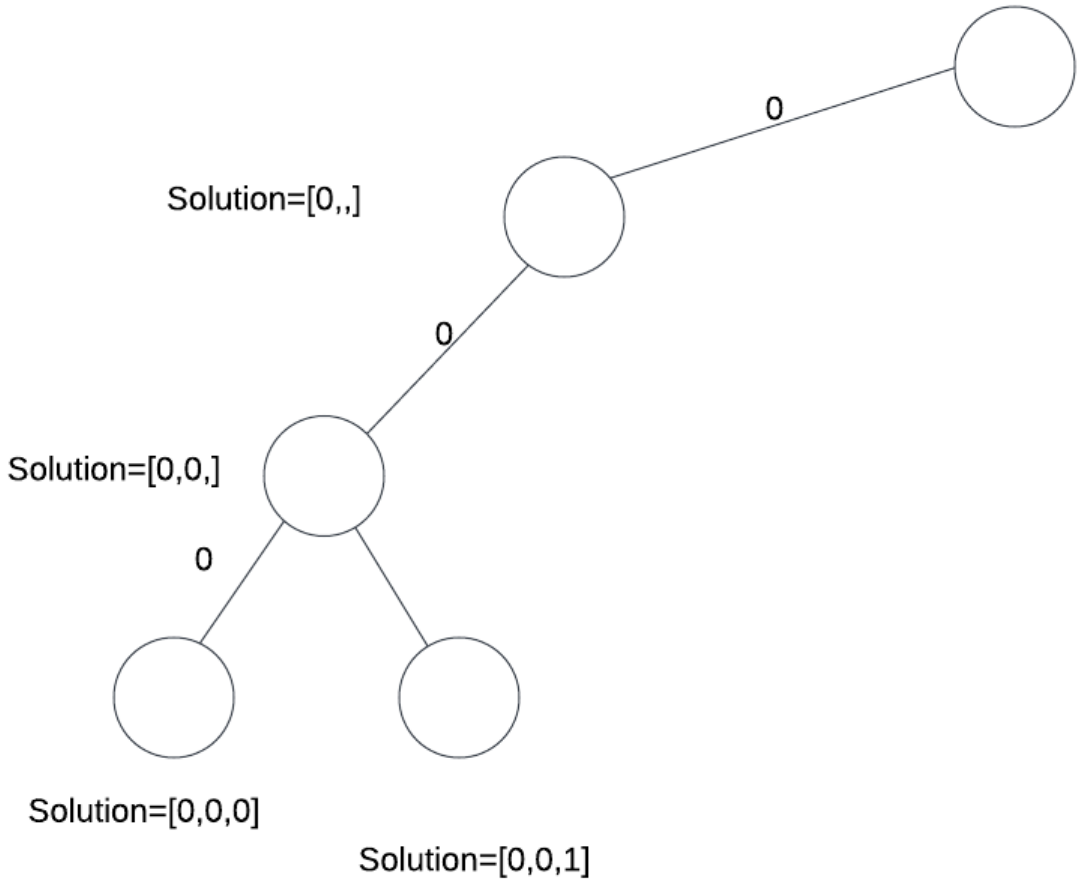
Brute Force Algorithms Steps

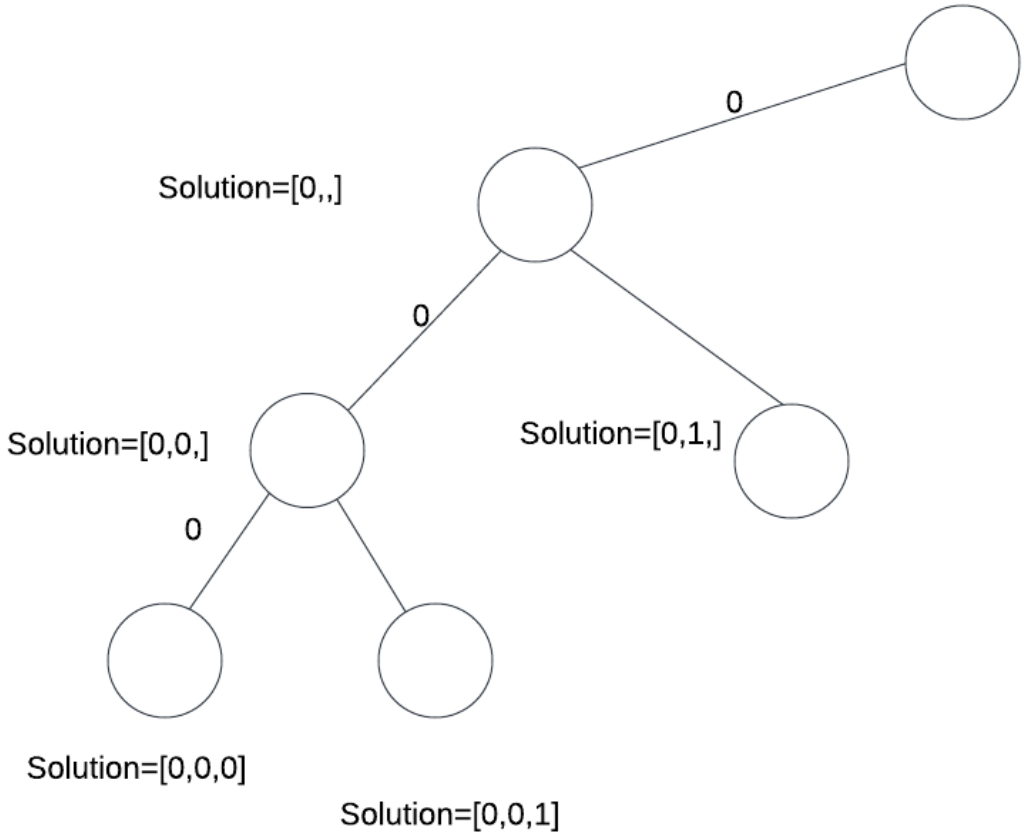
Solution=[0,,]

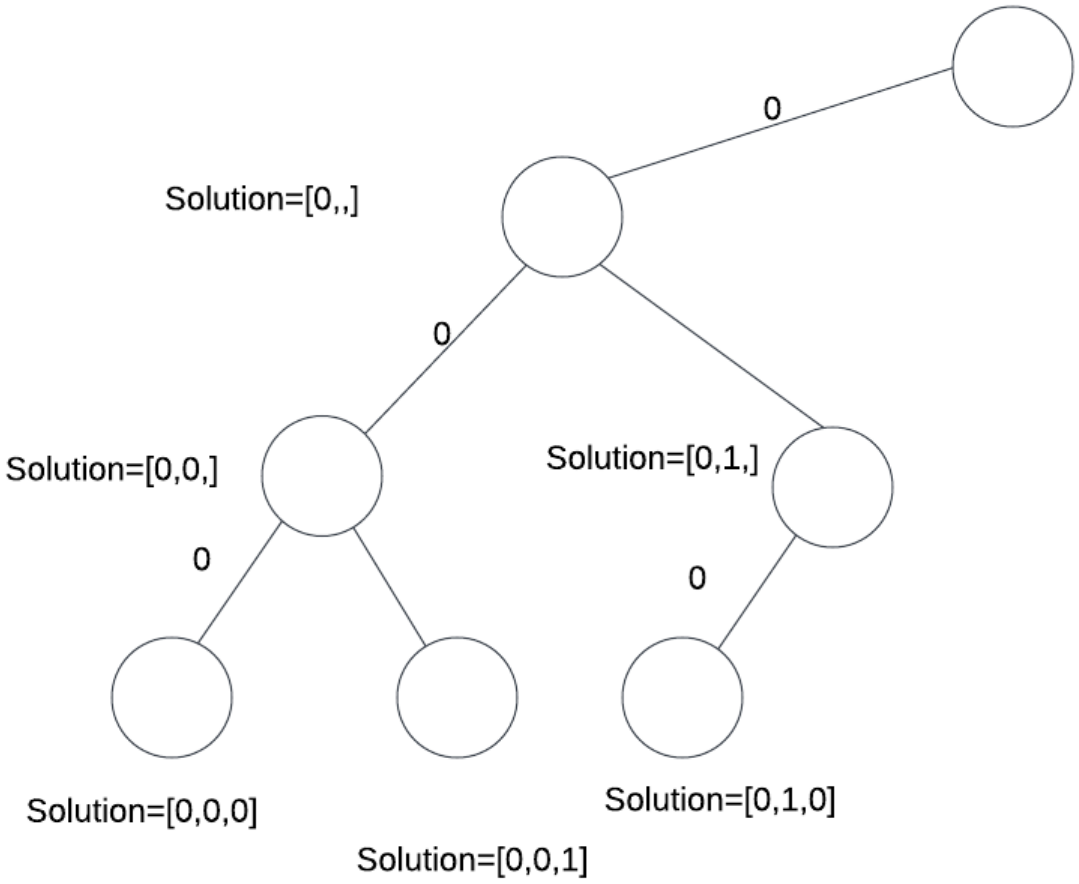


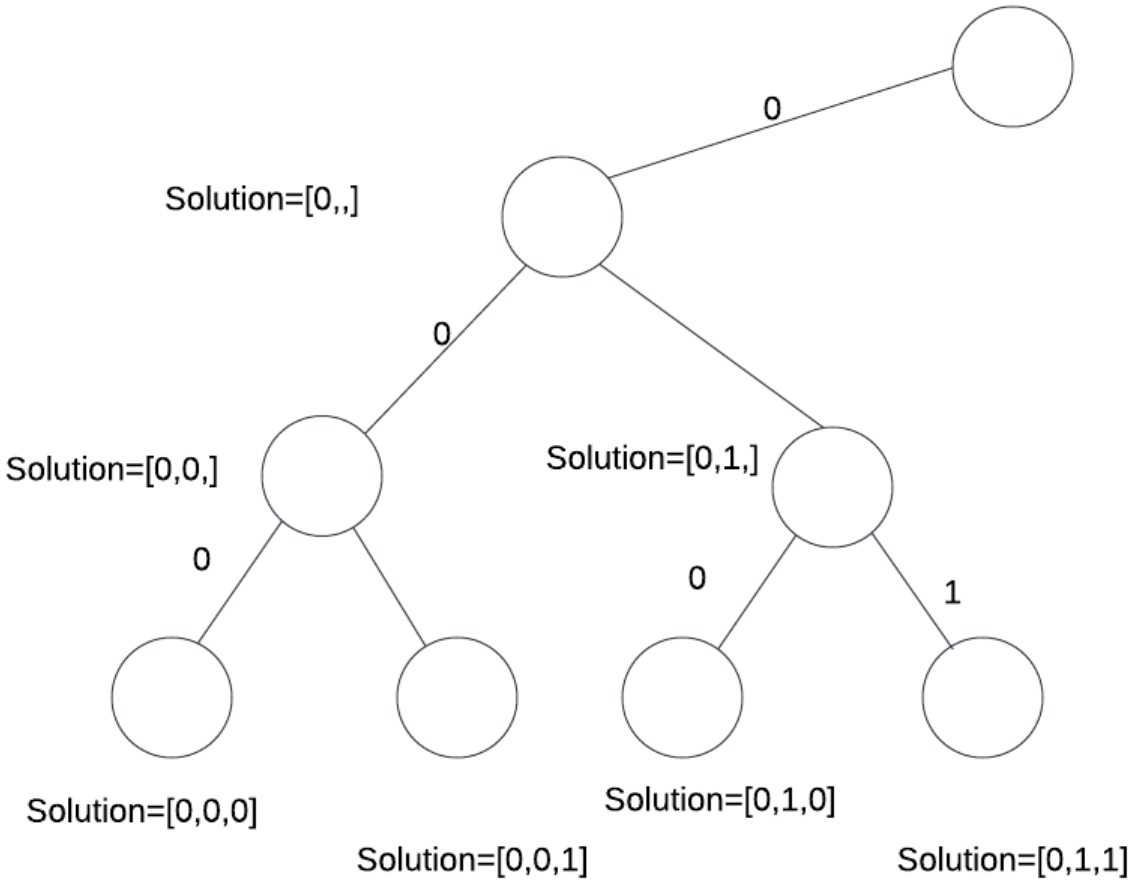

0
Solution=[0,,]
0
Solution=[0,0,]

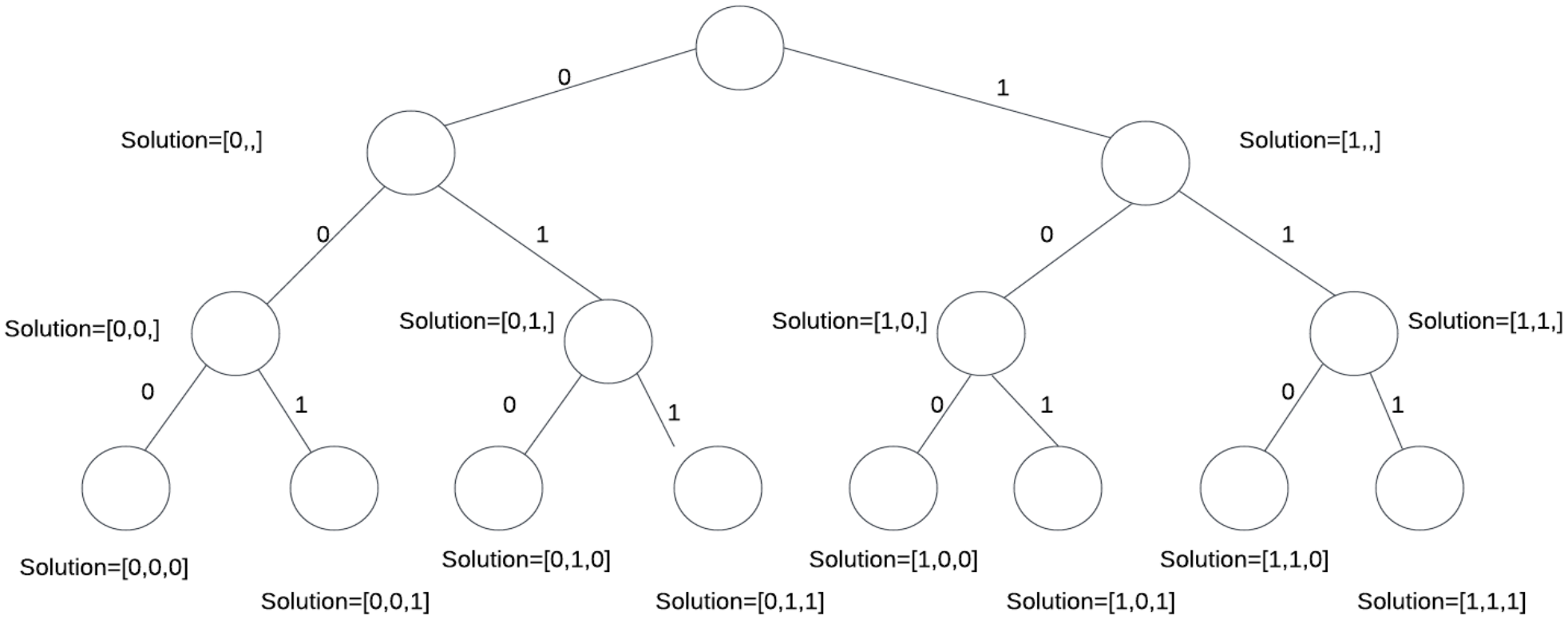

0
Solution=[0,,]
0
Solution=[0,0,]
Solution=[0,0,0]


0
Solution=[0,,]
0
Solution=[0,0,]
0
Solution=[0,0,0]
Solution=[0,0,1]


0
Solution=[0,,]
0
Solution=[0,0,]
Solution=[0,1,]
0
Solution=[0,0,0]
Solution=[0,0,1]


0
Solution=[0,,]
0
Solution=[0,0,]
Solution=[0,1,]
0
0
Solution=[0,0,0]
Solution=[0,0,1]
Solution=[0,1,0]


0
Solution=[0,,]
0
Solution=[0,0,]
Solution=[0,1,]
0
0
1
Solution=[0,0,0]
Solution=[0,1,0]
Solution=[0,0,1]
Solution=[0,1,1]


0
1
Solution=[0,,]
Solution=[1,,]
0
1
0
1
Solution=[0,0,]
Solution=[0,1,]
Solution=[1,0,]
Solution=[1,1,]
0
1
0
1
0
1
0
1
Solution=[0,0,0]
Solution=[0,0,1]
Solution=[0,1,0]
Solution=[0,1,1]
Solution=[1,0,0]
Solution=[1,0,1]
Solution=[1,1,0]
Solution=[1,1,1]

Brute Force: Ex 2 Subset Sum

```
Global
int N
Define integer array solution[N]
Define integer array values[N]
Define SUM
Define Total
```

```
bool isValidSolution()
   int s = 0
   for i = 0 to  N-1
        s += values[i] * solution[i]
   return s == SUM
```

```
void solve(int i)
   if (i == N)
       if isValidSolution()
           Total++;
      return

   solution[i] = 0
   solve(i + 1)

   solution[i] = 1
   solve(i + 1)
```

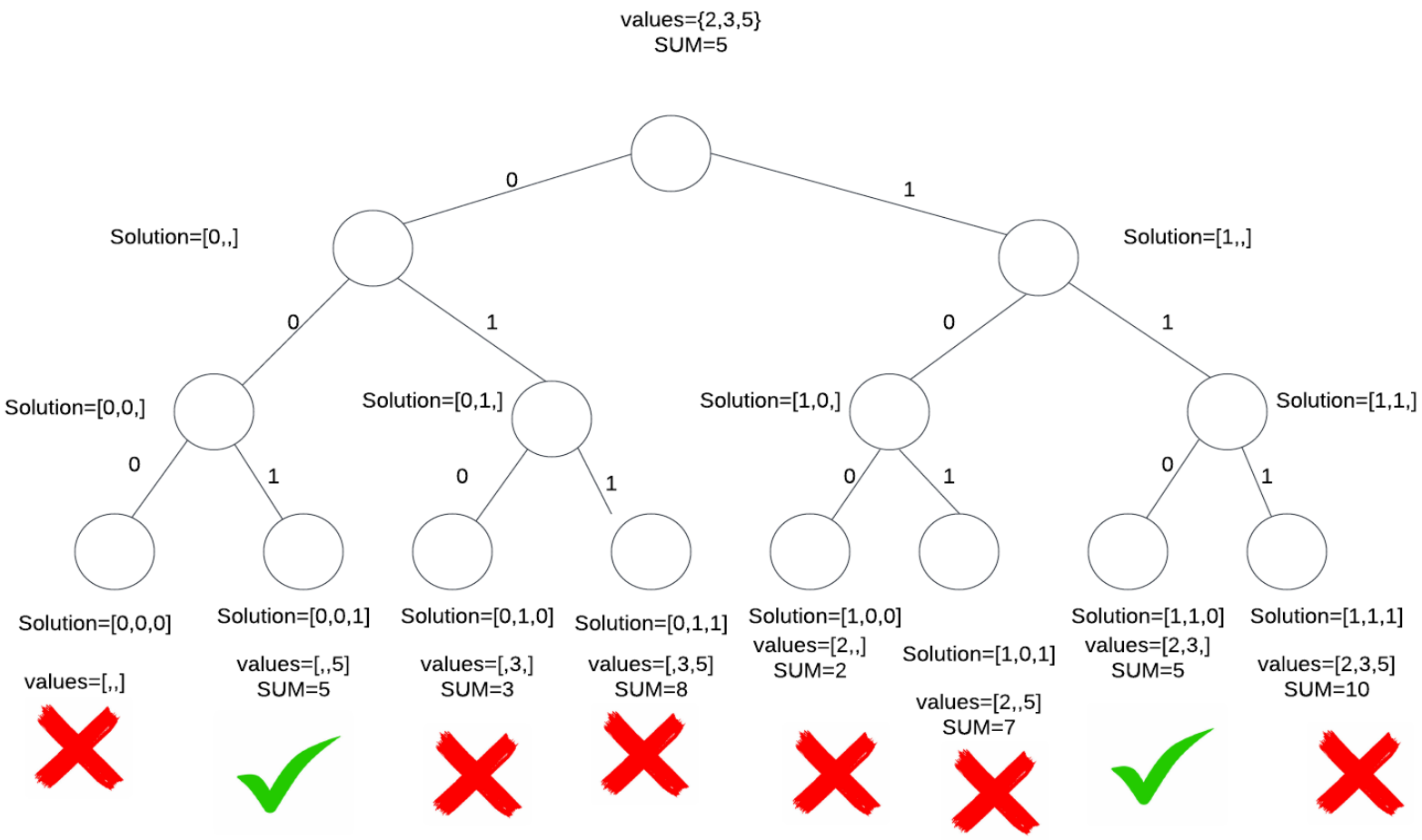
```
N=3
Values={2,3,5}
Solve(0)
```

Brute Force : Ex 2 Subset Sum

N=3
Values={2,3,5}
Solve(N)

Visited
Solution (8)
000
001
010
011
100
101
110
111

Total Valid=2



Brute Force: Ex 3 Password Crack

```
int N;
string solution="      "
string SECRET_PASSWORD = "ABCCED";
canAuthenticate()
   return solution == SECRET_PASSWORD;
```

```
solve(int i)
   if (i == N)
      if (canAuthenticate())
         print();
      Trials++;
      return;

   for char c = ‘A’ to 'Z'
      solution[i] = c
      solve(i + 1)
```

N = 6;

solve(0)

Print "Total number of trials is " + Trials

➔

ABCCED

Total number of trials if to visit all solutions is 308,915,776

26^6

Brute Force 0-1 Knapsack

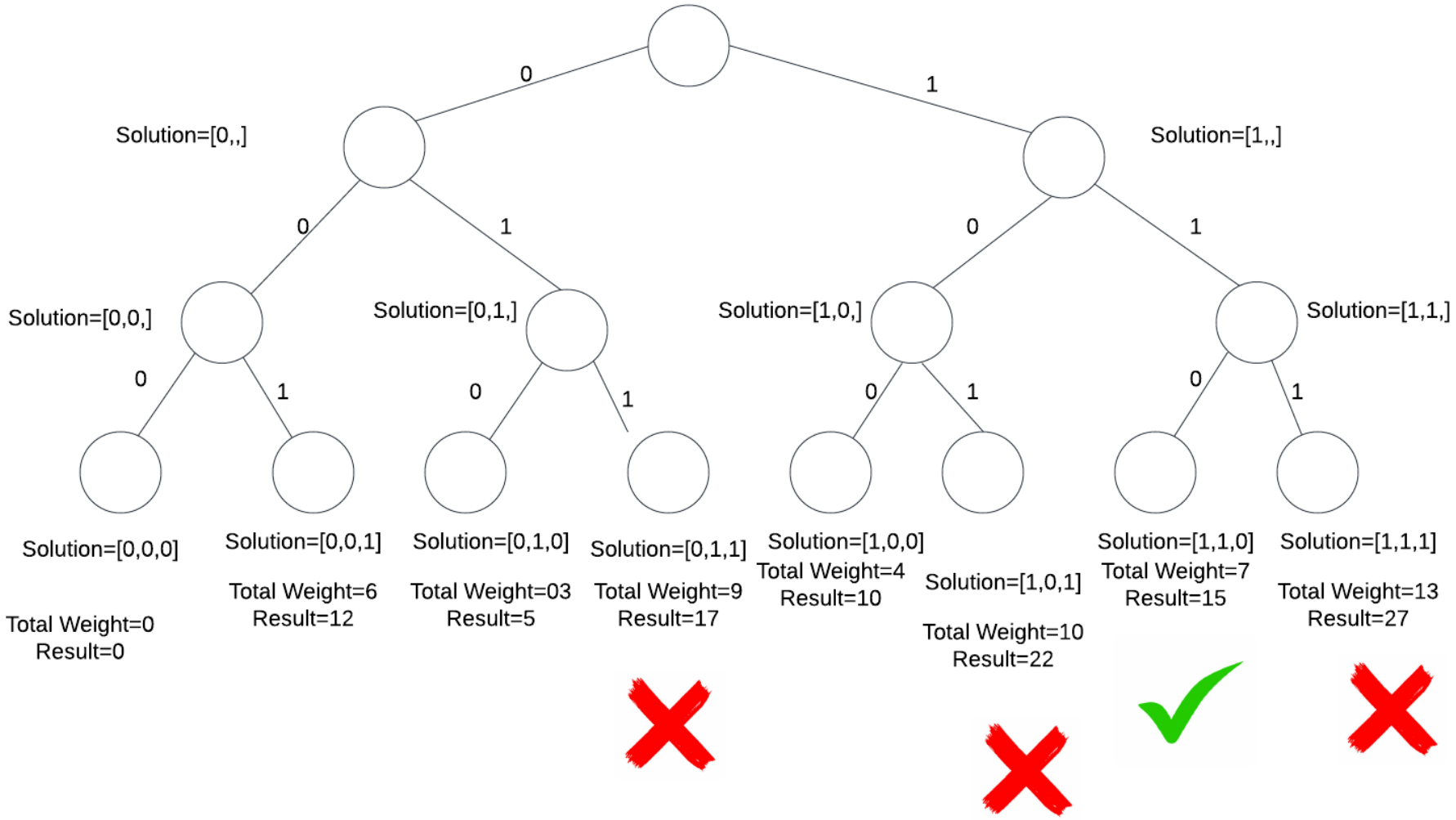
```
Global
// knapsack instance
int v[]     = {10, 5, 12}
int w[]     = {4, 3, 6}
int W       = 7
const int N = 3
int solution[N]
int MAX = -1
```

```
int get_value()
    int sum = 0
    for i = 0 to N-1
        sum += solution[i] * v[i]
    return sum
```

```
bool is_valid()
    int sum = 0
    for i = 0 to N-1
        sum += solution[i] * w[i]
    return sum <= W
```

```
void solve(int i)
    if (i == N)
        if is_valid()
            MAX = max(MAX, get_value())
        return;

    solution[i] = 0
    solve(i + 1)

    solution[i] = 1
    solve(i + 1)
```

knapsack instance
int v[] = {10, 5, 12}
int w[] = {4, 3, 6}
int W = 7



Brute Force Collecting Apples

- Think of how to enumerate the solutions?!
- Solution array dimension?
- Solve function additional considerations

Backtracking

- Initially, a backtracking algorithm is similar to brute force as it uses a depth-first recursive to search for potential solution(s)
- As it is growing the solution answer, it keeps checking for the validity of the partially formed solution, if a newly selected partial solution is invalid, it back tracks and tries other paths of solutions.
- Can be used to:
  - Search for a valid solution or enumerate solutions
  - Enumerate solutions
  - Solve an optimization problem by finding the optimal solution among explored solutions
- In general, backtracking is more efficient than brute force and reduces the search space.
- In the worse case, Backtracking behave like brute force if all solutions happen to be feasible ones.

0-1 Knapsack using Backtracking

```
Global
// knapsack instance
int v[]     = {10, 5, 12}
int w[]     = {4, 3, 6}
int W       = 7
const int N = 3
int solution[N]
int MAX = -1
```

```
int get_value()
    int sum = 0
    for i = 0 to N-1
        sum += solution[i] * v[i]
    return sum
```

```
bool is_valid(n)
    int sum = 0
    for i = 0 to n
        sum += solution[i] * w[i]
    return sum <= W
```

```
void solve(int i)
    if (i == N)
        if is_valid(i)
            MAX = max(MAX, get_value())
        return;

    solution[i] = 0
    solve(i + 1)

    solution[i] = 1
    if is_valid(i)
        solve(i + 1)
```

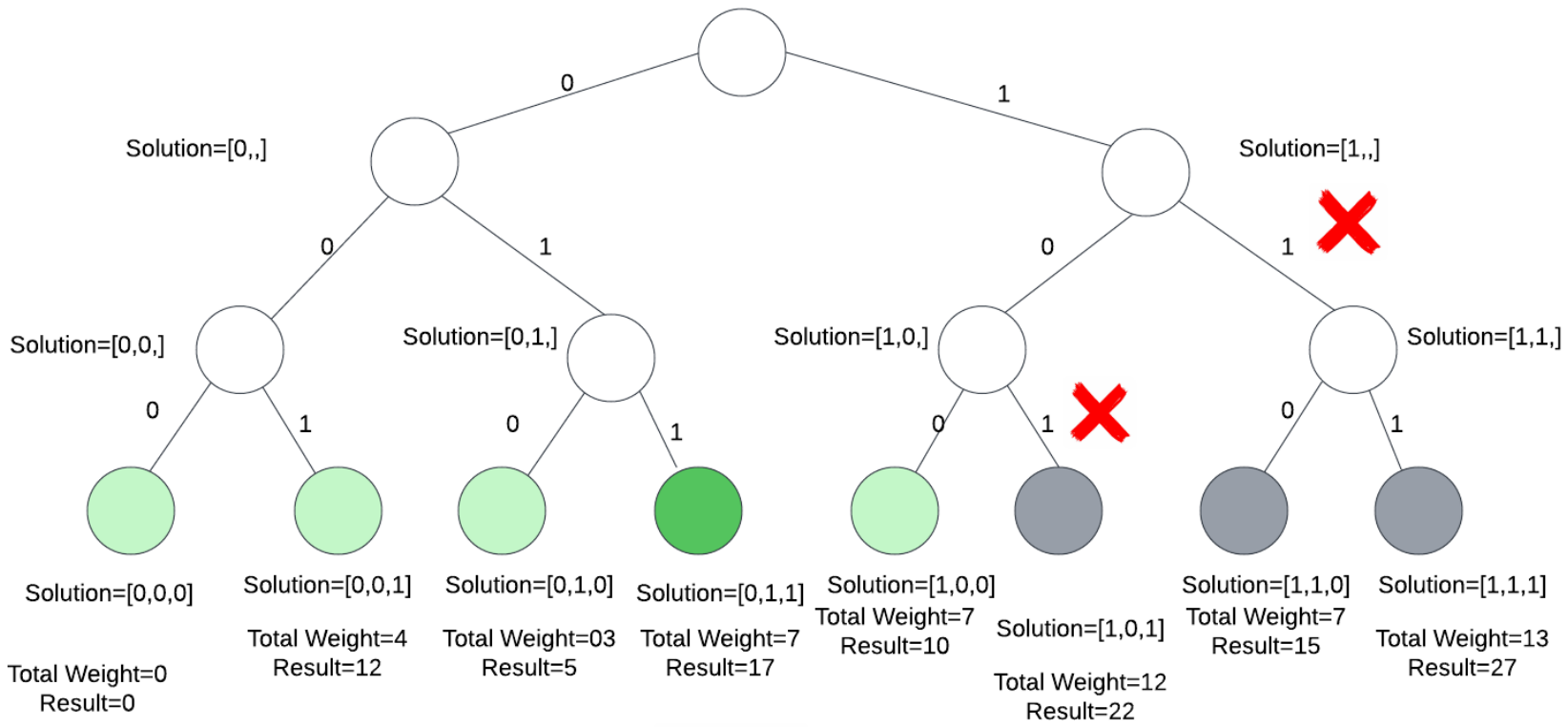
Solution not explored

Solution explored but not optimal

Solution explored and optimal

knapsack instance
int v[] = {10, 5, 12}
int w[] = {7, 3, 4}
int W = 7

3 out of 8 potential solutions eliminated and not explored using back tracking



3/8 options eliminated in this example

Weighted Activity Selection using Backtracking

```
Global
// WA instance
double s[] = {…}
double f[] = {…}
int    v[] = {…}
const int N = ..
int solution[N]
int MAX = -1
```

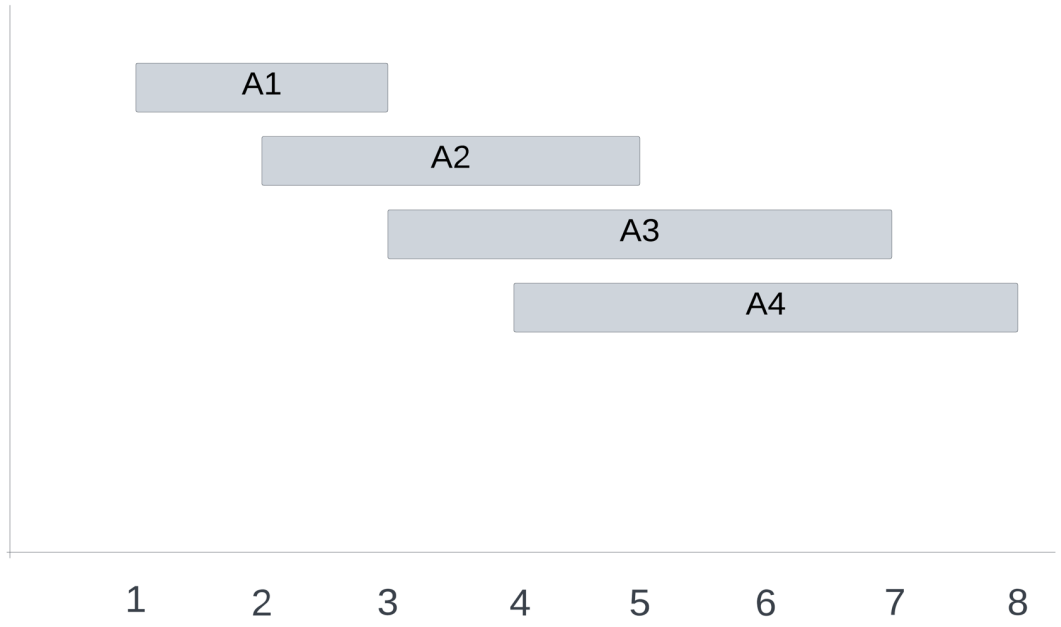
```
int get_value()
   int sum = 0
   for i = 0 to N-1
      sum += solution[i] * v[i]
   return sum
```

```
bool can_take(int n) {
     last_taken = -1;
   for i = n-1  to 0
      if (solution[i] == 1)
            last_taken = i
            break;
   if (last_taken == -1)
      return true

   return s[n] >= f[last_taken]
```

```
void solve(int i)
   if (i == N)
      if is_valid(i)
         MAX = max(MAX, get_value())
      return

   solution[i] = 0
   solve(i + 1)

   solution[i] = 1
   if is_valid(i)
      solve(i + 1)
```

Consider the Following Example

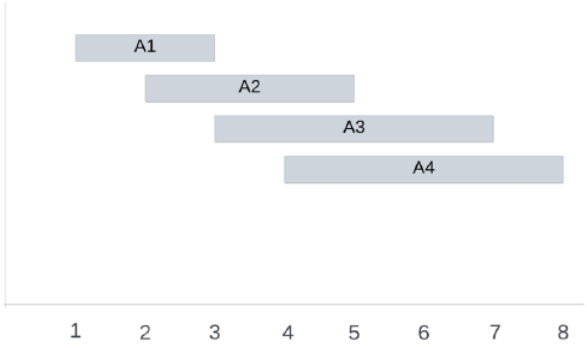
	A1	A2	A3	A4
S	1	2	3	4
F	3	5	7	8
W	5	4	8	10

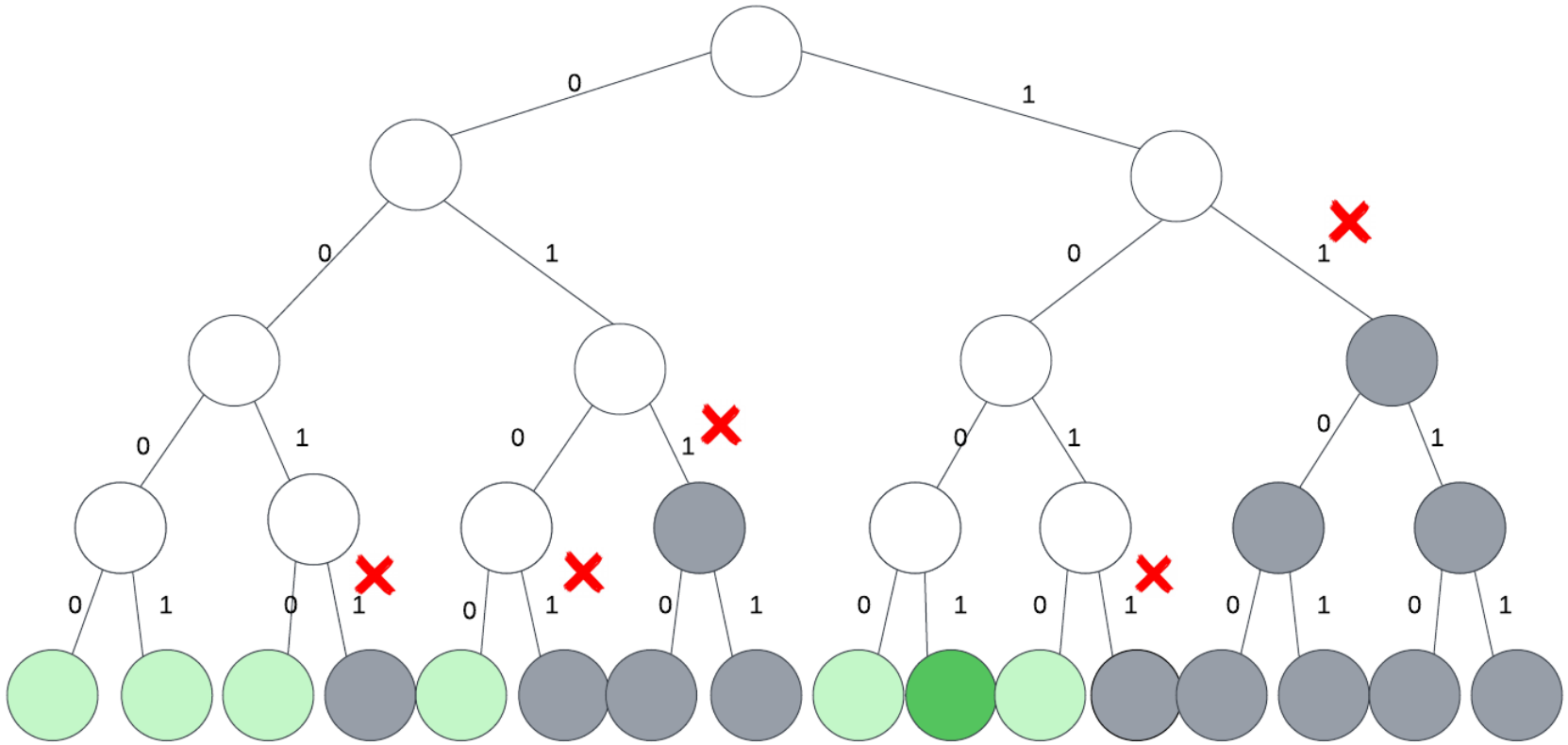


Solution not explored

Solution explored but not optimal

Solution explored and optimal



	A1	A2	A3	A4
S	1	2	3	4
F	3	5	7	8
W	5	4	8	10

9 out of 16 potential solutions eliminated and not explored using backtracking